

46. Event Loop: Microtasks aur Macrotasks

Browser aur Node.js dono ka JS execution **event loop** par chalta hai.

Event Loop kya hai?

Ek endless loop: engine tasks ka wait karta hai, unhe chalata hai, phir sota hai (~zero CPU) jab tak naya task na aaye.

1. Jab tak tasks hain, sabse **purana** pehle chalao.
2. Task nahi to sote raho, aane par phir se 1.

Tasks ke examples: external script load hui (chalao), `mousemove` (handlers chalao), `setTimeout` ka time aaya (callback chalao).

Engine busy ho to naye tasks **queue** me lagte hain (**macrotask queue**), "first come, first served".

Do baatein:

1. **Rendering task ke dauran kabhi nahi hoti.** Task kitna bhi lamba ho, DOM changes uske **khatam hone ke baad** hi dikhte hain.
2. Task bahut lamba ho to browser doosre kaam (user events) nahi kar sakta, aur "Page Unresponsive" warning aati hai.

Use case 1: CPU-heavy task ko todna

Heavy loop (jaise `1` se `1e9` tak ginti) page ko **hang** kar deta hai. Isse chhote tukdon me todo, beech me `setTimeout` se event loop ko "hawa" do:

```
js
let i = 0;
function count() {
  do { i++; } while (i % 1e6 !== 0);

  if (i == 1e9) alert("Done");
  else setTimeout(count); // agla tukda schedule
}
count();
```

Ab UI kaam karta rehta hai. Total time lagbhag utna hi.

Improvement: `setTimeout` ka schedule kaam se pehle karo. Nested `setTimeout` par browser me kam se kam ~4ms ka delay hota hai, isliye jitna jaldi schedule karo utna tez.

```
js
```

```
function count() {
  if (i < 1e9 - 1e6) setTimeout(count); // pehle schedule
  do { i++; } while (i % 1e6 != 0);
  if (i == 1e9) alert("Done");
}
```

Use case 2: Progress indication

DOM changes task khatam hone par hi paint hote hain, to ek bade loop me `progress.innerHTML = i` ka beech ka value nahi dikhega (sirf aakhri). Tukdon me todne par har tukde ke beech paint hota hai, isse progress bar dikhta hai.

Use case 3: Event ke baad kuch karna

Event handler me kuch kaam tab tak taalna ho jab tak event upar tak bubble ho ke poora handle na ho jaye, to use zero-delay `setTimeout` me daalo:

```
js
menu.onclick = function() {
  setTimeout(() => menu.dispatchEvent(new CustomEvent("menu-open", { bubbles: t
});
```

Macrotasks vs Microtasks

- **Macrotasks:** script, events, `setTimeout` (upar wale).
- **Microtasks:** sirf hamare code se, aam taur par **promises** (`.then/catch/finally`) ka handler), aur `await` ke andar bhi. Direct: `queueMicrotask(func)`.

Rule: Har macrotask ke turant baad, engine microtask queue ke saare tasks chalata hai, kisi aur macrotask, rendering ya kisi bhi cheez se pehle.

```
js
setTimeout(() => alert("timeout"));
Promise.resolve().then(() => alert("promise"));
alert("code");
// Order: code -> promise -> timeout
```

Microtasks ke beech UI ya network event handling nahi hoti, isliye environment same rehta hai (mouse coordinates nahi badalte).

`queueMicrotask(count)` progress bar me use karo to render aakhir me hota hai, sync code ki tarah (beech me nahi dikhta).

Poora algorithm

1. Macrotask queue se sabse purana task chalao (jaise "script").
2. **Saare microtasks** chalao (jab tak microtask queue khaali na ho).
3. Agar changes hain to **render** karo.
4. Macrotask queue khaali ho to naye macrotask ka wait karo.
5. Step 1 par jao.

Naya macrotask: zero-delay `setTimeout(f)`. Naya microtask: `queueMicrotask(f)` ya promise handlers.

Web Workers

Bahut heavy calculations jo event loop ko block na karein: **Web Workers** (alag parallel thread). Inka apna event loop aur variables hote hain, main thread se messages exchange karte hain. **DOM ka access nahi**, to mostly calculations ke liye, multiple CPU cores use karne ke liye.

Task: Output kya aayega?

```
js
console.log(1);
setTimeout(() => console.log(2));
Promise.resolve().then(() => console.log(3));
Promise.resolve().then(() => setTimeout(() => console.log(4)));
Promise.resolve().then(() => console.log(5));
setTimeout(() => console.log(6));
console.log(7);
```

Jawab: `1 7 3 5 2 6 4`

- `1`, `7` seedhe (sync).
- Fir microtasks: `3`, `4` ka `setTimeout` (macrotask queue ke end me), `5`.
- Fir macrotasks: `2`, `6`, `4`.